

Homework Part A — Sessions 1, 2 and 3 (MILK10k)

Course: Computer Vision and Speech Recognition — EADA **Student:** Kristof Papp

0. Setup

```
In [1]: import io
import os
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

warnings.filterwarnings("ignore") # hide library warnings so the PDF stays readable

# ----- configuration: the ONLY place where paths and the seed are set -----
DATA_DIR = Path(os.environ.get("MILK10K_DIR", "../data/milk10k"))
IMG_DIR = DATA_DIR / "images"
OUT_DIR = Path("../outputs") # generated files (CSV tables) go here
OUT_DIR.mkdir(exist_ok=True)
SEED = 42
EXPECTED_LESIONS = 5240
# -----
```

```
In [2]: # Load the two label files
meta = pd.read_csv(DATA_DIR / "metadata.csv") # one row per IMAGE
gt = pd.read_csv(DATA_DIR / "supplements" / "training_gt.csv") # one row per LESION (one-hot)

CLASSES = [c for c in gt.columns if c != "lesion_id"] # the 11 class names
gt["dx"] = gt[CLASSES].idxmax(axis=1) # one-hot -> class name
```

```
# Short name for the two image types: "derm" or "clinical"
meta["view"] = np.where(meta["image_type"].str.contains("dermoscop", case=False), "derm", "clinical")

# Find every image file once (works whether images are in one folder or in sub-folders)
image_paths = {p.stem: p for p in IMG_DIR.rglob("*.jpg")}
meta["path"] = meta["isic_id"].map(image_paths)

print("metadata.csv:", meta.shape, "| training_gt.csv:", gt.shape)
print("classes:", CLASSES)
print("image types:", meta["image_type"].value_counts().to_dict())
print(f"images found on disk: {meta['path'].notna().sum()} of {len(meta)}")
```

```
metadata.csv: (10480, 19) | training_gt.csv: (5240, 13)
classes: ['AKIEC', 'BCC', 'BEN_OTH', 'BKL', 'DF', 'INF', 'MAL_OTH', 'MEL', 'NV', 'SCCKA', 'VASC']
image types: {'dermoscopic': 5240, 'clinical: close-up': 5240}
images found on disk: 10480 of 10480
```

A1. Session 1 — Trusting and shaping the data

A1.1 Cross-check the two label files

```
In [3]: # (a) the set of lesion_id is identical in both files
ids_meta, ids_gt = set(meta["lesion_id"]), set(gt["lesion_id"])
print("(a) lesions only in metadata:", len(ids_meta - ids_gt), "| only in gt:", len(ids_gt - ids_meta))
assert ids_meta == ids_gt, "lesion_id sets differ!"

# (b) every lesion has exactly ONE positive class
n_positive = gt[CLASSES].sum(axis=1)
print("(b) lesions without exactly one positive class:", (n_positive != 1).sum())
assert (n_positive == 1).all(), "some lesions have 0 or >1 positive classes"
```

```
(a) lesions only in metadata: 0 | only in gt: 0
(b) lesions without exactly one positive class: 0
```

```
In [4]: # (c) does each of the 11 classes map to exactly ONE diagnosis_1 value?
lesion_level = meta.drop_duplicates("lesion_id").merge(gt[["lesion_id", "dx"]], on="lesion_id")
```

```
class_map = pd.crosstab(lesion_level["dx"], lesion_level["diagnosis_1"]) # lesion counts
display(class_map)

n_targets = (class_map > 0).sum(axis=1)
not_unique = class_map[n_targets > 1]
print("(c) classes that map to MORE than one diagnosis_1 value:")
print(not_unique if len(not_unique) else "none")
```

diagnosis_1 Benign Indeterminate Malignant

dx			
AKIEC	0	123	180
BCC	0	0	2522
BEN_OTH	44	0	0
BKL	544	0	0
DF	52	0	0
INF	50	0	0
MAL_OTH	0	0	9
MEL	0	0	450
NV	746	0	0
SCCKA	0	0	473
VASC	47	0	0

(c) classes that map to MORE than one diagnosis_1 value:

```
diagnosis_1 Benign Indeterminate Malignant
dx
AKIEC      0      123      180
```

```
In [5]: # (d) each diagnosis_3 belongs to one diagnosis_2, and each diagnosis_2 to one diagnosis_1
def check_hierarchy(df, child, parent):
    """Print every value of `child` that appears with more than one value of `parent`."""
    sub = df.dropna(subset=[child])
```

```

n_parents = sub.groupby(child)[parent].nunique()
bad = n_parents[n_parents > 1]
print(f"(d) {child} values with more than one {parent}: {len(bad)}")
if len(bad):
    print(sub[sub[child].isin(bad.index)].groupby([child, parent]).size())
return bad

```

```

bad_3_2 = check_hierarchy(lesion_level, "diagnosis_3", "diagnosis_2")
bad_2_1 = check_hierarchy(lesion_level, "diagnosis_2", "diagnosis_1")

```

(d) diagnosis_3 values with more than one diagnosis_2: 0

(d) diagnosis_2 values with more than one diagnosis_1: 0

```

In [6]: # (e) the two images of a lesion have identical age, sex, site and diagnosis fields
check_cols = ["age_approx", "sex", "anatom_site_general"] + [c for c in meta.columns if c.startswith("diagnosis")]
n_unique = meta.groupby("lesion_id")[check_cols].nunique(dropna=False)
violations = (n_unique > 1).sum()
print("(e) lesions whose two images disagree, per column:")
print(violations.to_string())
assert (violations == 0).all(), "the two images of some lesions disagree"

```

(e) lesions whose two images disagree, per column:

```

age_approx      0
sex             0
anatom_site_general  0
diagnosis_1     0
diagnosis_2     0
diagnosis_3     0
diagnosis_4     0
diagnosis_confirm_type  0

```

Answer A1.1. Checks (a), (b), (d) and (e) all pass. In (c), 10 of the 11 classes map to exactly one diagnosis_1 value, but **AKIEC does not**: 180 AKIEC lesions are Malignant and 123 are Indeterminate. All 123 Indeterminate lesions of the dataset are AKIEC. So diagnosis_1 cannot be derived from an 11-class prediction: a model that says "AKIEC" does not tell us whether the lesion is Malignant or Indeterminate. For the project, the 3-class target must be trained directly on the diagnosis_1 column. On any new medical dataset I would always repeat (a) the ID match between files, (b) exactly one label per sample and (e) consistency between images of the same lesion. The most dangerous failure would be (a) or (e): labels silently attached to the wrong lesion, so the model learns and is evaluated on wrong ground truth without any error message.

A1.2 Ask the data five questions

```
In [7]: # Q1. BCC lesions in patients aged 70+ on the head/neck
print("site values:", lesion_level["anatom_site_general"].value_counts(dropna=False).to_dict())
is_head_neck = lesion_level["anatom_site_general"].str.contains("head", case=False, na=False)
q1 = lesion_level[(lesion_level["dx"] == "BCC") & (lesion_level["age_approx"] >= 70) & is_head_neck]
print("Q1. BCC lesions, age >= 70, head/neck:", len(q1))
```

site values: {nan: 1956, 'head/neck': 1491, 'lower extremity': 1046, 'upper extremity': 739, 'oral/genital': 8}
 Q1. BCC lesions, age >= 70, head/neck: 380

```
In [8]: # Q2. Class with the highest % of images with image_manipulation == "altered"
print("image_manipulation values:", meta["image_manipulation"].value_counts(dropna=False).to_dict())
images = meta.merge(gt[["lesion_id", "dx"]], on="lesion_id")
pct_altered = images["image_manipulation"].eq("altered").groupby(images["dx"]).mean() * 100
print(f"Q2. {pct_altered.idxmax()} has the highest share of altered images: {pct_altered.max():.1f}%")
```

image_manipulation values: {'instrument only': 10145, 'altered': 335}
 Q2. BEN_OTH has the highest share of altered images: 18.2%

```
In [9]: # Q3. Youngest and oldest patient with melanoma
mel_age = lesion_level.loc[lesion_level["dx"] == "MEL", "age_approx"]
print(f"Q3. MEL: youngest {mel_age.min():.0f}, oldest {mel_age.max():.0f}, gap {mel_age.max() - mel_age.min():.0f} years")
```

Q3. MEL: youngest 20, oldest 85, gap 65 years

```
In [10]: # Q4. Is a missing anatom_site_general informative?
has_site = lesion_level["anatom_site_general"].notna().map({True: "with site", False: "without site"})
dist = pd.crosstab(lesion_level["dx"], has_site, normalize="columns") * 100
if "without site" in dist.columns:
    dist["difference (pp)"] = dist["without site"] - dist["with site"]
    display(dist.round(1))
    biggest = dist["difference (pp)"].abs().idxmax()
    print(f"Q4. Class that changes most: {biggest} ({dist.loc[biggest, 'difference (pp)']:.1f} percentage points)")
else:
    print("Q4. No lesion has a missing site.")
```

anatom_site_general	with site	without site	difference (pp)
dx			
AKIEC	8.0	2.0	-6.0
BCC	47.4	49.3	1.9
BEN_OTH	0.9	0.8	-0.1
BKL	10.6	10.1	-0.5
DF	1.3	0.5	-0.8
INF	1.0	0.9	-0.1
MAL_OTH	0.1	0.3	0.2
MEL	7.2	11.0	3.8
NV	10.1	21.1	11.0
SCCKA	12.7	2.9	-9.8
VASC	0.7	1.2	0.4

Q4. Class that changes most: NV (+11.0 percentage points)

```
In [11]: # Q5. % Malignant per diagnosis_confirm_type
q5 = lesion_level.groupby("diagnosis_confirm_type").agg(
    n_lesions=("lesion_id", "size"),
    pct_malignant=("diagnosis_1", lambda s: (s == "Malignant").mean() * 100),
).sort_values("pct_malignant", ascending=False)
display(q5.round(1))
```

	n_lesions	pct_malignant
diagnosis_confirm_type		
histopathology	5016	72.3
single contributor clinical assessment	224	2.2

Answer Q4. A missing site is informative, not random. NV changes the most: it is 21.1% of the lesions without a site but only 10.1% of those with a site (+11.0 pp), while SCCA drops from 12.7% to 2.9% (−9.8 pp) and AKIEC from 8.0% to 2.0% (−6.0 pp). Missing sites probably come from specific contributors or protocols, so "site is missing" could act as a shortcut for NV and must be handled carefully (e.g. an explicit "unknown" category, checked for bias).

Answer Q5. 5,016 of the 5,240 lesions (96%) were confirmed by histopathology, and 72.3% of them are malignant. The 224 lesions confirmed by a single contributor's clinical assessment are only 2.2% malignant. This shows the dataset was built mainly from lesions suspicious enough to be biopsied, so malignant lesions are heavily over-represented compared with daily practice. The confirmation type is a result of the diagnostic process, so it must never be a model input.

A1.3 Build a lesion-level table

```
In [12]: # one row per lesion: the two isic_ids side by side (pivot, no loops)
wide = meta.pivot(index="lesion_id", columns="view", values="isic_id")
wide = wide.rename(columns={"derm": "derm_id", "clinical": "clinical_id"})
wide.columns.name = None

info = meta.groupby("lesion_id")[["diagnosis_1", "age_approx", "sex", "anatom_site_general"]].first()

lesions = (wide.join(info)
            .join(gt.set_index("lesion_id")["dx"])
            .reset_index()
            .rename(columns={"age_approx": "age", "anatom_site_general": "site"}))
lesions = lesions[["lesion_id", "derm_id", "clinical_id", "diagnosis_1", "dx", "age", "sex", "site"]]

assert len(lesions) == EXPECTED_LESIONS, f"expected {EXPECTED_LESIONS} rows, got {len(lesions)}"
assert lesions["derm_id"].notna().all() and lesions["clinical_id"].notna().all()
assert lesions["lesion_id"].is_unique

lesions.to_csv(OUT_DIR / "lesions.csv", index=False)
print(lesions.shape)
display(lesions.head())
```

(5240, 8)

	lesion_id	derm_id	clinical_id	diagnosis_1	dx	age	sex	site
0	IL_0000652	ISIC_4671410	ISIC_8149219	Malignant	BCC	70.0	male	head/neck
1	IL_0003176	ISIC_5371928	ISIC_3904045	Malignant	BCC	45.0	female	head/neck
2	IL_0004688	ISIC_3624913	ISIC_0791494	Malignant	BCC	50.0	male	lower extremity
3	IL_0005081	ISIC_5186409	ISIC_5667730	Malignant	SCCKA	45.0	male	head/neck
4	IL_0006177	ISIC_1048297	ISIC_8803389	Malignant	BCC	75.0	male	upper extremity

A1.4 Task framing

Input: the dermoscopic image of a skin lesion (optionally also the clinical close-up). Metadata such as age and site could be added later, but never fields produced by the diagnosis process (confirmation type, diagnosis_2–4). **Output:** a probability for Benign / Malignant / Indeterminate for each lesion. **User and moment:** a dermatologist or GP during the consultation, after taking the dermoscopic photo and before deciding whether to biopsy or refer. It is a decision-support tool, not a replacement for the doctor. **Metric:** recall (sensitivity) for Malignant, together with balanced accuracy / macro-F1, because the classes are imbalanced and plain accuracy hides errors on small classes. **Costly error:** a false negative (a malignant lesion called benign) is far more costly than a false positive, which only causes an extra biopsy. **Differences from real use:** (1) MILK10k is enriched with biopsied, suspicious lesions, so malignant lesions are far more common than in a normal clinic; (2) the images were taken with specific devices and by experts, while real users may have different cameras, lighting and skin types.

A2. Session 2 — Working with pixels

A2.1 Arrays, views and memory

```
In [13]: sample_id = meta.loc[meta["path"].notna() & (meta["view"] == "derm"), "isic_id"].iloc[0]
pil_img = Image.open(image_paths[sample_id]).convert("RGB")
arr = np.array(pil_img)
```

```

print("image", sample_id, "shape", arr.shape, arr.dtype)

# (a) NumPy-only operations (indexing / axis swaps)
mine = {
    "left-right flip": arr[:, ::-1],          # reverse the column axis
    "up-down flip":    arr[::-1, :],          # reverse the row axis
    "rotate 90°":      arr.transpose(1, 0, 2)[::-1], # swap rows/cols, then flip up-down = 90° counter-clockwise
    "transpose":       arr.transpose(1, 0, 2),    # swap rows and columns
}

pillow_ops = {
    "left-right flip": Image.Transpose.FLIP_LEFT_RIGHT,
    "up-down flip":    Image.Transpose.FLIP_TOP_BOTTOM,
    "rotate 90°":      Image.Transpose.ROTATE_90,
    "transpose":       Image.Transpose.TRANSPOSE,
}

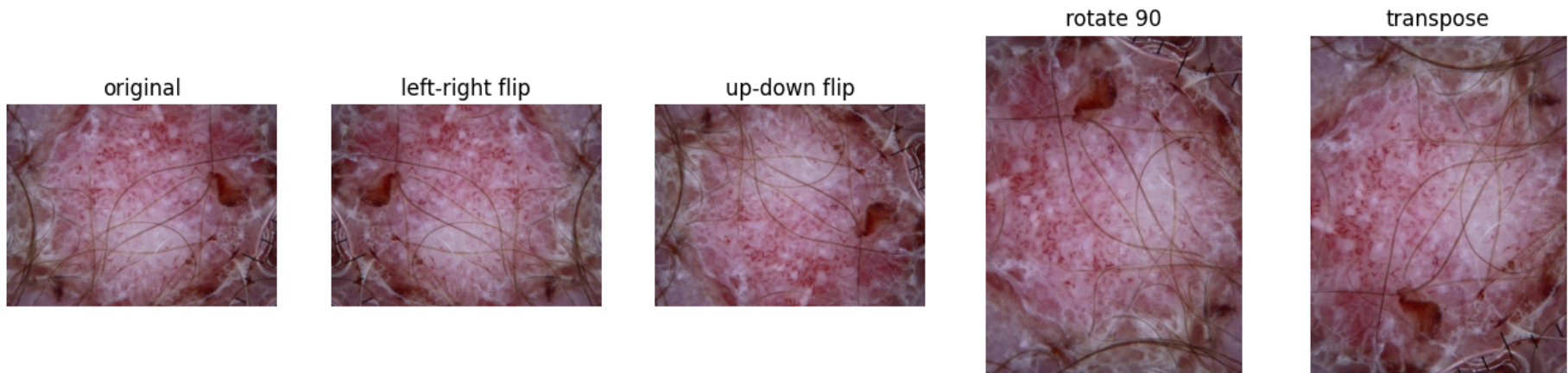
fig, axes = plt.subplots(1, 5, figsize=(16, 3.5))
axes[0].imshow(arr); axes[0].set_title("original")
for ax, (name, result) in zip(axes[1:], mine.items()):
    reference = np.array(pil_img.transpose(pillow_ops[name]))
    identical = np.array_equal(result, reference)
    is_view = np.shares_memory(result, arr) # (b) view or copy?
    print(f"{name:16s} identical to Pillow: {identical} | {'VIEW' if is_view else 'COPY'}")
    assert identical
    ax.imshow(result); ax.set_title(name)
for ax in axes:
    ax.axis("off")
plt.show()

```

```

image ISIC_0051817 shape (450, 600, 3) uint8
left-right flip  identical to Pillow: True | VIEW
up-down flip     identical to Pillow: True | VIEW
rotate 90        identical to Pillow: True | VIEW
transpose        identical to Pillow: True | VIEW

```



```
In [14]: # (b) two-line experiment: changing a VIEW changes the original
demo = arr.copy()
flipped = demo[:, ::-1]; flipped[0, 0] = [255, 0, 0] # paint the first pixel of the VIEW red
print("original pixel at the mirrored position:", demo[0, -1], "-> the original changed too!")

safe = demo[:, ::-1].copy() # .copy() makes an independent array
print("after .copy(), shares memory:", np.shares_memory(safe, demo))
```

original pixel at the mirrored position: [255 0 0] -> the original changed too!
after .copy(), shares memory: False

Answer A2.1(b). All four NumPy operations return **views**: they only change how NumPy walks through the same memory (strides), no pixels are copied. Modifying a view therefore modifies the original image (the experiment above). Pillow always returns a new image, so `np.array(...)` of it is a copy. To get an independent array from NumPy you must call `.copy()`.

```
In [15]: # (c) memory budget
def image_size(path):
    """Return (width, height) by reading only the file header (no decoding)."""
    with Image.open(path) as im:
        return im.size

sizes = np.array([image_size(p) for p in meta["path"].dropna()])
avg_pixels = (sizes[:, 0] * sizes[:, 1]).mean()
n_images = len(meta)
print(f"average original size: {sizes[:, 0].mean():.0f} x {sizes[:, 1].mean():.0f} pixels")
```

```
rows = []
for dtype, n_bytes in [("uint8", 1), ("float32", 4)]:
    rows.append({"dtype": dtype,
                 "original resolution (GB)": n_images * avg_pixels * 3 * n_bytes / 1e9,
                 "224x224 (GB)": n_images * 224 * 224 * 3 * n_bytes / 1e9})
display(pd.DataFrame(rows).round(2))
```

average original size: 600 x 450 pixels

	dtype	original resolution (GB)	224x224 (GB)
0	uint8	8.49	1.58
1	float32	33.96	6.31

Answer A2.1(c). At the original resolution (about 600×450) all 10,480 images need about 8.5 GB as uint8 and 34 GB as float32, which does not fit in a typical laptop. At 224×224 they need 1.6 GB (uint8) or 6.3 GB (float32): possible, but it would use most of the RAM. Instead I load images lazily from disk, one batch at a time, inside a Dataset/DataLoader, and convert to float only for the current batch.

A2.2 JPEG compression and a shortcut hiding in the file size

```
In [16]: def psnr(original, compressed):
    """Peak signal-to-noise ratio in dB for 8-bit images."""
    mse = np.mean((original.astype(np.float64) - compressed.astype(np.float64)) ** 2)
    return float("inf") if mse == 0 else 10 * np.log10(255 ** 2 / mse)

results, decoded = [], {}
for quality in [95, 75, 50, 25, 10]:
    buffer = io.BytesIO()
    pil_img.save(buffer, format="JPEG", quality=quality)    # encode in memory
    size_kb = buffer.tell() / 1024
    buffer.seek(0)
    decoded[quality] = np.array(Image.open(buffer).convert("RGB"))
    results.append({"quality": quality, "size (KB)": round(size_kb, 1),
                   "PSNR (dB)": round(psnr(arr, decoded[quality]), 2)})
display(pd.DataFrame(results))
```

```
# zoomed crop (64x64 pixels from the centre)
h, w = arr.shape[:2]
crop = (slice(h // 2 - 32, h // 2 + 32), slice(w // 2 - 32, w // 2 + 32))
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, (title, image) in zip(axes, [("original", arr), ("quality 95 (best)", decoded[95]), ("quality 10 (worst)", decoded[10])]):
    ax.imshow(image[crop], interpolation="nearest")
    ax.set_title(title); ax.axis("off")
plt.show()
```

	quality	size (KB)	PSNR (dB)
0	95	73.0	51.23
1	75	38.4	65.35
2	50	30.0	35.85
3	25	17.4	33.22
4	10	9.4	29.24

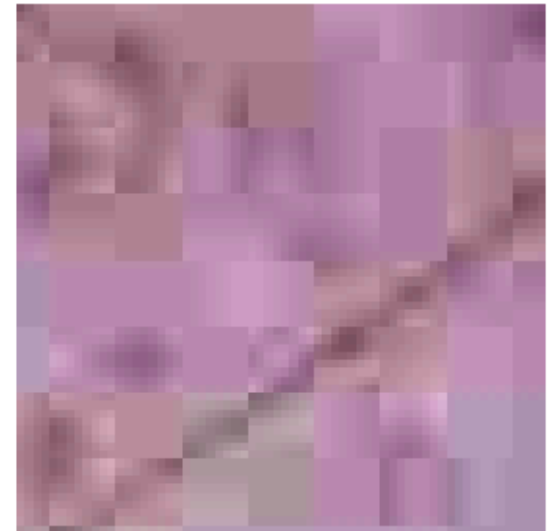
original



quality 95 (best)



quality 10 (worst)



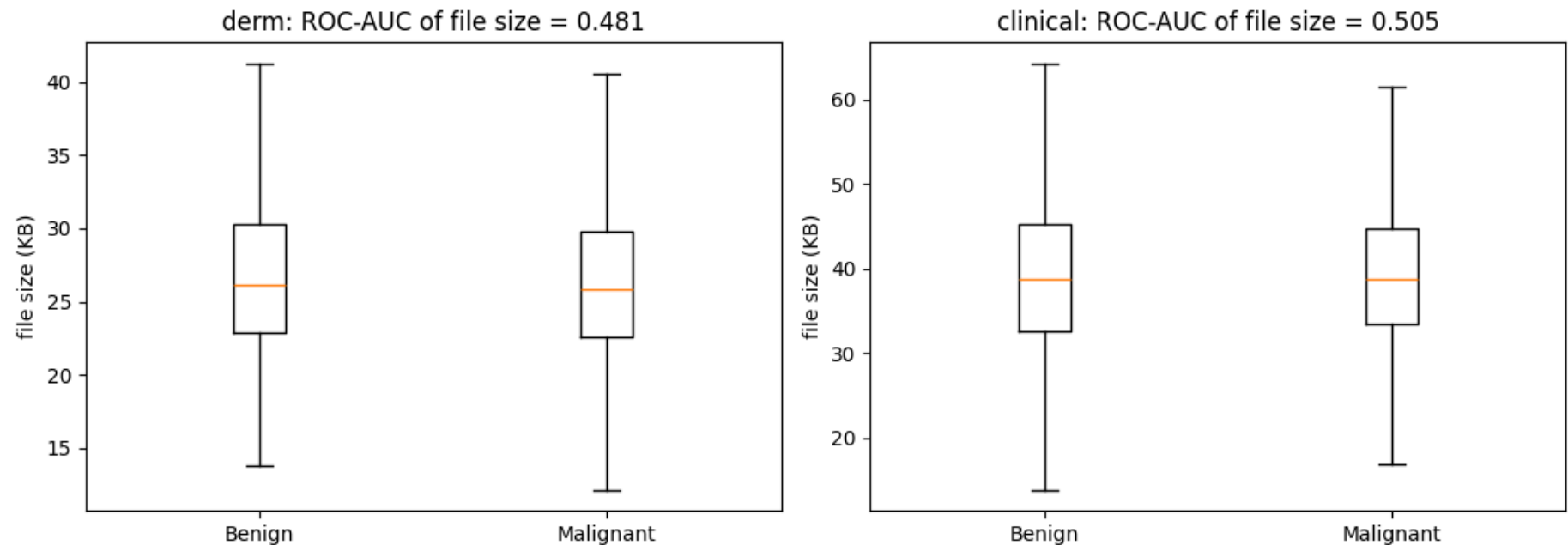
Answer A2.2(b). The MILK10k images are already JPEG, so they already contain compression artefacts (8×8 blocks, blurred fine detail, colour bleeding). Saving them again adds a second, different set of artefacts (double compression), and resizing after compression can smooth or exaggerate those blocks. The table above shows it: quality 75 gives a *higher* PSNR (65 dB) than quality 95 (51 dB). The original file was most likely saved at a quality close to 75, so re-encoding with the same settings almost reproduces it, while quality 95 uses different quantisation and adds new rounding errors. A model could learn these artefacts instead of the lesion itself, especially if the compression level differs between hospitals or classes. So I should decode once, resize once, and never re-save images as JPEG inside the pipeline.

```
In [17]: # (c) shortcut check: file size (os.stat, no decoding)
from sklearn.metrics import roc_auc_score

meta["file_kb"] = meta["path"].map(lambda p: os.stat(p).st_size / 1024, na_action="ignore")
bm = meta[meta["diagnosis_1"].isin(["Benign", "Malignant"]) & meta["file_kb"].notna()]

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, view in zip(axes, ["derm", "clinical"]):
    sub = bm[bm["view"] == view]
    groups = [sub.loc[sub["diagnosis_1"] == c, "file_kb"] for c in ["Benign", "Malignant"]]
    ax.boxplot(groups, showfliers=False)
    ax.set_xticks([1, 2], ["Benign", "Malignant"])
    auc = roc_auc_score(sub["diagnosis_1"] == "Malignant", sub["file_kb"])
    ax.set_title(f"{view}: ROC-AUC of file size = {auc:.3f}")
    ax.set_ylabel("file size (KB)")
    print(f"{view:8s} AUC = {auc:.3f} (0.5 = no information; far from 0.5 in either direction = shortcut)")
plt.tight_layout(); plt.show()
```

derm AUC = 0.481 (0.5 = no information; far from 0.5 in either direction = shortcut)
clinical AUC = 0.505 (0.5 = no information; far from 0.5 in either direction = shortcut)



Answer A2.2(c). The ROC-AUC of file size is 0.481 for dermoscopic and 0.505 for clinical images, both very close to 0.5 (random guessing), and the boxplots overlap almost completely. So in MILK10k there is no meaningful file-size shortcut. It was still worth checking: a shortcut like this can appear through acquisition factors such as different cameras or dermatoscopes per clinic, different resolutions or JPEG quality settings, or suspicious lesions being photographed more carefully before a biopsy. A CNN could pick these up, so resizing and preprocessing must stay identical for all images.

A3. Session 3 — Splits, imbalance, leakage, augmentation, datasets

A3.1 Measure the leak

```
In [18]: from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
```

```

from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import GroupKFold, StratifiedGroupKFold, StratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder

NUM_FEATURES = ["age_approx"]
CAT_FEATURES = ["sex", "anatom_site_general", "image_type"]

def make_model(seed):
    """Random forest on metadata; imputation and encoding live INSIDE the pipeline, so they are fitted on train only."""
    preprocess = ColumnTransformer([
        ("num", SimpleImputer(strategy="median"), NUM_FEATURES),
        ("cat", make_pipeline(SimpleImputer(strategy="constant", fill_value="missing"),
                             OneHotEncoder(handle_unknown="ignore")), CAT_FEATURES),
    ])
    return Pipeline([("prep", preprocess),
                     ("rf", RandomForestClassifier(n_estimators=200, min_samples_leaf=1, random_state=seed, n_jobs=-1))])

def stratified_group_folds(table, label_col, n_splits, seed):
    """StratifiedGroupKFold folds as (train_positions, val_positions) in `table`, grouped by lesion_id.

    The rows are shuffled with `seed` first and the splitter is run with shuffle=False. Reason: with
    scikit-learn < 1.8 the built-in shuffle=True balanced the classes much worse (we measured a 3.6 pp
    spread between folds instead of ~0.1 pp).
    """
    order = np.random.default_rng(seed).permutation(len(table))
    shuffled = table.iloc[order]
    sgkf = StratifiedGroupKFold(n_splits=n_splits)
    return [(order[tr], order[va]) for tr, va in sgkf.split(shuffled, shuffled[label_col], groups=shuffled["lesion_id"])]

def naive_split(df, seed):
    """Image-level split, stratified on diagnosis_1 (LEAKY: the two images of a lesion can be separated)."""
    return train_test_split(df.index, test_size=0.2, stratify=df["diagnosis_1"], random_state=seed)

def grouped_split(df, seed):
    """Split grouped by lesion_id: both images of a lesion end up on the same side."""
    train_pos, test_pos = stratified_group_folds(df, "diagnosis_1", 5, seed)[0]
    return df.index[train_pos], df.index[test_pos]

def sibling_oracle(df, train_idx, test_idx):
    """Predict a test image with the label of its sibling image if that sibling is in train, else the train majority class."""

```

```

train, test = df.loc[train_idx], df.loc[test_idx]
label_in_train = train.groupby("lesion_id")["diagnosis_1"].first()
majority = train["diagnosis_1"].mode()[0]
sibling_in_train = test["lesion_id"].isin(label_in_train.index)
prediction = test["lesion_id"].map(label_in_train).fillna(majority)
return balanced_accuracy_score(test["diagnosis_1"], prediction), sibling_in_train.mean() * 100

data = meta.reset_index(drop=True)          # metadata only: no images needed here
rows = []
for seed in range(5):
    for split_name, split_fn in [("(a) naive image-level", naive_split), ("(b) grouped by lesion", grouped_split)]:
        train_idx, test_idx = split_fn(data, seed)
        model = make_model(seed).fit(data.loc[train_idx, NUM_FEATURES + CAT_FEATURES], data.loc[train_idx, "diagnosis_1"])
        pred = model.predict(data.loc[test_idx, NUM_FEATURES + CAT_FEATURES])
        oracle_bacc, pct_sibling = sibling_oracle(data, train_idx, test_idx)
        rows.append({"split": split_name, "seed": seed,
                     "RF balanced acc": balanced_accuracy_score(data.loc[test_idx, "diagnosis_1"], pred),
                     "oracle balanced acc": oracle_bacc, "% test images with sibling in train": pct_sibling})

leak = pd.DataFrame(rows)
summary = leak.groupby("split")[["RF balanced acc", "oracle balanced acc", "% test images with sibling in train"]].agg(["mean"])
display(summary.round(3))

```

	RF balanced acc		oracle balanced acc		% test images with sibling in train	
	mean	std	mean	std	mean	std
split						
(a) naive image-level	0.426	0.004	0.847	0.022	79.79	1.211
(b) grouped by lesion	0.412	0.000	0.333	0.000	0.00	0.000

Answer A3.1(d). The leak barely inflated the random forest: balanced accuracy is 0.426 ± 0.004 with the naive split and 0.412 ± 0.000 with the grouped split, a difference of only about 1.4 points. The two images of a lesion only share age, sex and site, and the same combination is shared by many other lesions, so a metadata model cannot "recognise" a lesion and gains nothing from seeing its sibling. The sibling oracle shows the real potential of the leak: in the naive split about 80% of the test images have their sibling in train, and copying its label gives a balanced accuracy of about 0.85 (vs 0.33 and 0% in the grouped split). A CNN that can recognise the same lesion from its pixels could exploit

exactly this and look much better than it really is. So "I measured no inflation" with a weak model proves nothing about a stronger model: the grouped split is required by design.

A3.2 A reusable three-way split

```
In [19]: def split_lesions(lesions, val_size=0.2, test_size=0.2, seed=42, label_col="dx"):
        """Split a LESION table into train/val/test lists of lesion_id.

        Uses StratifiedGroupKFold twice: first one fold is taken as test (test_size=0.2 -> 5 folds),
        then one fold of the remaining lesions is taken as validation (0.2 of the total = 1/4 of the rest).
        Groups = lesion_id, stratified on `label_col`.
        """

        n_test_folds = round(1 / test_size)
        rest_pos, test_pos = stratified_group_folds(lesions, label_col, n_test_folds, seed)[0]
        rest = lesions.iloc[rest_pos]

        n_val_folds = round((1 - test_size) / val_size)
        train_pos, val_pos = stratified_group_folds(rest, label_col, n_val_folds, seed)[0]

        return (rest.iloc[train_pos]["lesion_id"].tolist(),
                rest.iloc[val_pos]["lesion_id"].tolist(),
                lesions.iloc[test_pos]["lesion_id"].tolist())
```

```
In [20]: # property test over 10 seeds
VAL_SIZE, TEST_SIZE = 0.2, 0.2
RARE = [c for c in ["MAL_OTH", "DF", "INF", "VASC", "BEN_OTH"] if c in CLASSES]
dx_of = lesions.set_index("lesion_id")["dx"]
presence = {c: 0 for c in RARE}

for seed in range(10):
    train_ids, val_ids, test_ids = split_lesions(lesions, VAL_SIZE, TEST_SIZE, seed)
    tr, va, te = set(train_ids), set(val_ids), set(test_ids)
    # no overlap
    assert not (tr & va) and not (tr & te) and not (va & te), f"overlap with seed {seed}"
    # sizes within +-1 percentage point
    n = len(lesions)
    assert abs(len(va) / n - VAL_SIZE) <= 0.01, f"val size off with seed {seed}"
    assert abs(len(te) / n - TEST_SIZE) <= 0.01, f"test size off with seed {seed}"
```

```

# reproducibility
assert split_lesions(lesions, VAL_SIZE, TEST_SIZE, seed) == (train_ids, val_ids, test_ids)
# rare classes present in val AND test?
for c in RARE:
    if (dx_of[val_ids] == c).any() and (dx_of[test_ids] == c).any():
        presence[c] += 1

print("All property tests passed for 10 seeds.")
print("Seeds (out of 10) where the class is in BOTH val and test:")
display(pd.DataFrame({"lesions in dataset": lesions["dx"].value_counts()[RARE], "present in val AND test": pd.Series(presence)

```

All property tests passed for 10 seeds.

Seeds (out of 10) where the class is in BOTH val and test:

	lesions in dataset	present in val AND test
MAL_OTH	9	10
DF	52	10
INF	50	10
VASC	47	10
BEN_OTH	44	10

Answer A3.2. DF, INF, VASC and BEN_OTH (44–52 lesions) are present in val and test for every seed, with about 9–10 lesions each. MAL_OTH has only 9 lesions, so val and test get only 1–2 MAL_OTH lesions each. Thanks to stratification it was present in both in all 10 seeds, but only just. Any metric on that class (recall, F1) is therefore extremely noisy: one prediction more or less changes it by 50–100 points. Results for the rarest class should be reported with that caveat, or the class should be merged/evaluated with cross-validation over many folds.

A3.3 Why StratifiedGroupKFold?

```
In [21]: image_table = meta.merge(gt[["lesion_id", "dx"]], on="lesion_id").reset_index(drop=True)
```

```

def fold_report(name, table, folds):
    """Leaked lesions, largest class-proportion spread and folds without MAL_OTH for a list of (train, val) folds."""
    leaked, proportions = set(), []
    for train_pos, val_pos in folds:

```

```

leaked |= set(table.iloc[train_pos]["lesion_id"]) & set(table.iloc[val_pos]["lesion_id"])
proportions.append(table.iloc[val_pos]["dx"].value_counts(normalize=True).reindex(CLASSES, fill_value=0) * 100)
p = pd.DataFrame(proportions)
spread = p.max() - p.min()
return {"strategy": name, "leaked lesions": len(leaked),
        "largest spread (pp)": round(spread.max(), 2), "class with largest spread": spread.idxmax(),
        "folds without MAL_OTH": int((p["MAL_OTH"] == 0).sum()) if "MAL_OTH" in p else None}

strategies = [
    fold_report("(i) GroupKFold", lesions,
                list(GroupKFold(n_splits=5).split(lesions, groups=lesions["lesion_id"]))),
    fold_report("(ii) StratifiedKFold (image table)", image_table,
                list(StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED).split(image_table, image_table["dx"]))),
    fold_report("(iii) StratifiedGroupKFold", lesions,
                stratified_group_folds(lesions, "dx", 5, SEED)),
]
display(pd.DataFrame(strategies))

```

	strategy	leaked lesions	largest spread (pp)	class with largest spread	folds without MAL_OTH
0	(i) GroupKFold	0	3.72	BCC	0
1	(ii) StratifiedKFold (image table)	4158	0.05	AKIEC	0
2	(iii) StratifiedGroupKFold	0	0.19	BCC	0

Answer A3.3. StratifiedKFold on the image table has well-balanced folds but leaks 4,158 lesions (about 80%): their two images land in different folds, so it is not acceptable. GroupKFold has no leakage but ignores the classes, so class proportions differ by up to about 3.7 pp between folds (BCC) and a rare class could disappear from a fold. StratifiedGroupKFold gives both: zero leakage and class proportions almost as balanced as StratifiedKFold (largest spread 0.19 pp vs 0.05 pp), with MAL_OTH in every fold. This is the strategy I use.

Note: with my scikit-learn version, StratifiedGroupKFold's built-in `shuffle=True` balanced the classes badly (3.6 pp spread, one fold without MAL_OTH). Newer versions (≥ 1.8) fix this. My `stratified_group_folds()` shuffles the rows itself with the seed and runs the splitter with `shuffle=False`, which gives balanced folds on any version.

A3.4 Metrics and the cost of errors

```

In [22]: from sklearn.dummy import DummyClassifier
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score, confusion_matrix, f1_score

train_ids, val_ids, test_ids = split_lesions(lesions, VAL_SIZE, TEST_SIZE, SEED)
train_l = lesions[lesions["lesion_id"].isin(train_ids)]
test_l = lesions[lesions["lesion_id"].isin(test_ids)]
y_train, y_test = train_l["diagnosis_1"], test_l["diagnosis_1"]
LABELS = ["Benign", "Indeterminate", "Malignant"]

# cost matrix: rows = truth, columns = prediction
COST = pd.DataFrame([[0, 5, 1],      # truth Benign:      unnecessary work-up = 1, called Indeterminate = 5
                    [5, 0, 5],      # truth Indeterminate: any error = 5
                    [50, 5, 0]],    # truth Malignant:   missed = 50, called Indeterminate = 5
                    index=LABELS, columns=LABELS)

display(COST)

dummies = {
    "always-Malignant": DummyClassifier(strategy="constant", constant="Malignant"),
    "stratified-random": DummyClassifier(strategy="stratified", random_state=SEED),
    "uniform-random": DummyClassifier(strategy="uniform", random_state=SEED),
    "always-Benign": DummyClassifier(strategy="constant", constant="Benign"),
}
X_train, X_test = np.zeros((len(y_train), 1)), np.zeros((len(y_test), 1)) # dummies ignore the features

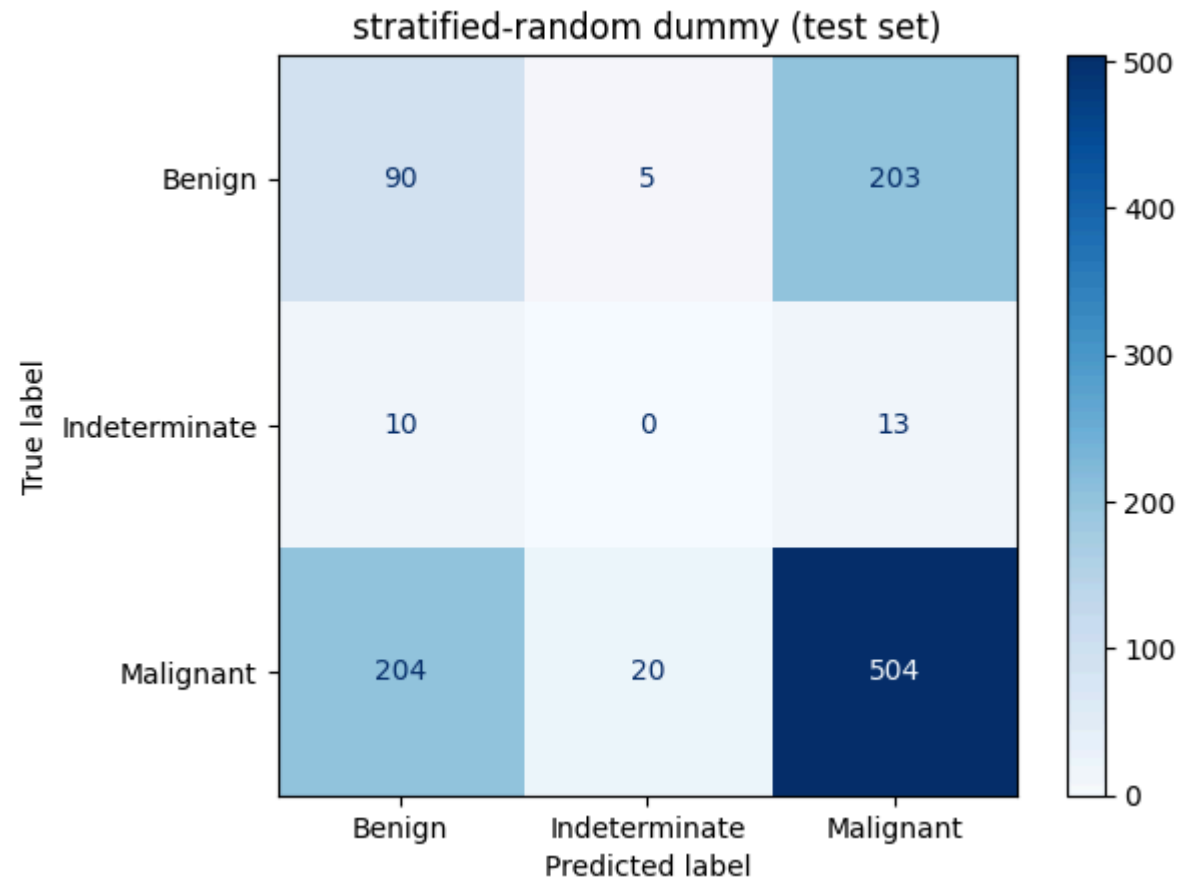
rows, matrices = [], {}
for name, clf in dummies.items():
    pred = clf.fit(X_train, y_train).predict(X_test)
    cm = confusion_matrix(y_test, pred, labels=LABELS)
    matrices[name] = cm
    rows.append({"predictor": name,
                "accuracy": accuracy_score(y_test, pred),
                "balanced acc": balanced_accuracy_score(y_test, pred),
                "macro-F1": f1_score(y_test, pred, labels=LABELS, average="macro", zero_division=0),
                "expected cost / lesion": (cm * COST.values).sum() / len(y_test)})
display(pd.DataFrame(rows).round(3))

ConfusionMatrixDisplay(matrices["stratified-random"], display_labels=LABELS).plot(cmap="Blues")
plt.title("stratified-random dummy (test set)"); plt.show()

```

	Benign	Indeterminate	Malignant
Benign	0	5	1
Indeterminate	5	0	5
Malignant	50	5	0

	predictor	accuracy	balanced acc	macro-F1	expected cost / lesion
0	always-Malignant	0.694	0.333	0.273	0.394
1	stratified-random	0.566	0.331	0.332	10.146
2	uniform-random	0.335	0.330	0.268	13.833
3	always-Benign	0.284	0.333	0.147	34.809



```
In [23]: # (c) which constant prediction is cheapest?
share = y_test.value_counts(normalize=True).reindex(LABELS, fill_value=0)
constant_cost = {f"always-{c}": (share * COST[c]).sum() for c in LABELS}
print("expected cost per lesion of each constant prediction:")
for name, cost in sorted(constant_cost.items(), key=lambda kv: kv[1]):
    print(f" {name:22s} {cost:.3f}")
```

```
expected cost per lesion of each constant prediction:
always-Malignant      0.394
always-Indeterminate  4.890
always-Benign        34.809
```

Answer A3.4(c). Always-Malignant is the cheapest constant prediction (0.39 per lesion, versus about 4.9 for always-Indeterminate and 35 for always-Benign), because missing a malignant lesion costs 50× more than an unnecessary work-up. It is still useless as a classifier: its balanced accuracy is only 0.33. In MILK10k the ranking by accuracy happens to agree with the ranking by cost, because Malignant is also the majority class (~70%), so the cheap guess is also the frequent one. In a real clinic, where malignant lesions are rare, the two rankings would disagree: always-Benign would have the highest accuracy and by far the highest cost. Balanced accuracy is about 0.33 for all four (none of them has any skill), and macro-F1 even ranks stratified-random first. The metric must reflect clinical consequences, not only the share of correct answers.

A3.5 Is this augmentation label-safe?

```
In [24]: import torch
from torchvision import transforms

def load_small(path, size=256):
    """Open a JPEG quickly at reduced size (draft mode) and return an RGB image of at most `size` pixels."""
    img = Image.open(path)
    img.draft("RGB", (size, size))
    img = img.convert("RGB")
    img.thumbnail((size, size))
    return img

def center_crop(img, fraction=0.5):
    """Keep the central part of the image. Simple proxy for 'lesion pixels': dermoscopic lesions are centred."""
    w, h = img.size
    dw, dh = int(w * (1 - fraction) / 2), int(h * (1 - fraction) / 2)
    return img.crop((dw, dh, w - dw, h - dh))

def circular_mean_deg(degrees):
    """Mean of angles in degrees (hue is an angle: 359° and 1° are close)."""
    rad = np.deg2rad(degrees)
    return np.rad2deg(np.arctan2(np.sin(rad).mean(), np.cos(rad).mean())) % 360

def angle_diff(a, b):
    """Smallest signed difference between two angles in degrees."""
    return (a - b + 180) % 360 - 180

def hue_and_brightness(img):
    """Circular-mean hue (degrees) and mean brightness V (0-1) of an RGB PIL image."""
```

```

hsv = np.array(img.convert("HSV")).astype(float)
hue_deg = hsv[..., 0] / 255 * 360
return circular_mean_deg(hue_deg.ravel()), hsv[..., 2].mean() / 255

derm = meta[(meta["view"] == "derm") & meta["path"].notna()]
crops, stats = {}, {}
for group in ["Benign", "Malignant"]:
    paths = derm[derm["diagnosis_1"] == group].sample(100, random_state=SEED)["path"]
    crops[group] = [center_crop(load_small(p)) for p in paths]
    stats[group] = np.array([hue_and_brightness(c) for c in crops[group]])

hue_gap = abs(angle_diff(circular_mean_deg(stats["Malignant"][:, 0]), circular_mean_deg(stats["Benign"][:, 0])))
v_gap = abs(stats["Malignant"][:, 1].mean() - stats["Benign"][:, 1].mean())
print(f"class gap: hue = {hue_gap:.2f} degrees | brightness V = {v_gap:.3f}")

```

class gap: hue = 6.59 degrees | brightness V = 0.074

```

In [25]: # change induced by ColorJitter (measured on 20 images x 5 random draws each)
torch.manual_seed(SEED)
test_imgs = crops["Benign"][:10] + crops["Malignant"][:10]
before = [hue_and_brightness(img) for img in test_imgs]

rows = []
for hue in [0.02, 0.05, 0.1, 0.5]:
    jitter = transforms.ColorJitter(hue=hue)
    changes = [abs(angle_diff(hue_and_brightness(jitter(img)))[0], b[0])) for img, b in zip(test_imgs, before) for _ in range(5)]
    rows.append({"augmentation": f"hue={hue} (max ±{hue*360:.0f}°)", "mean change": np.mean(changes),
                "class gap": hue_gap, "unit": "degrees"})
for brightness in [0.1, 0.3, 0.6]:
    jitter = transforms.ColorJitter(brightness=brightness)
    changes = [abs(hue_and_brightness(jitter(img))[1] - b[1]) for img, b in zip(test_imgs, before) for _ in range(5)]
    rows.append({"augmentation": f"brightness={brightness}", "mean change": np.mean(changes),
                "class gap": v_gap, "unit": "V (0-1)"})

audit = pd.DataFrame(rows)
audit["changes colour MORE than the class gap?"] = audit["mean change"] > audit["class gap"]
display(audit.round(3))

```

	augmentation	mean change	class gap	unit	changes colour MORE than the class gap?
0	hue=0.02 (max $\pm 7^\circ$)	3.193	6.590	degrees	False
1	hue=0.05 (max $\pm 18^\circ$)	8.897	6.590	degrees	True
2	hue=0.1 (max $\pm 36^\circ$)	16.876	6.590	degrees	True
3	hue=0.5 (max $\pm 180^\circ$)	85.817	6.590	degrees	True
4	brightness=0.1	0.027	0.074	V (0-1)	False
5	brightness=0.3	0.090	0.074	V (0-1)	True
6	brightness=0.6	0.162	0.074	V (0-1)	True

Answer A3.5. *Lesion pixels* were approximated with the central 50% of each dermoscopic image (lesions are centred in dermoscopy). The real class gap is small: 6.6° of hue and 0.074 of brightness (V). Hue jitter of 0.05, 0.1 and 0.5 shifts the hue more than that gap (8.9° , 16.9° , 85.8°), and so do brightness 0.3 and 0.6 (0.090 and 0.162). Only hue = 0.02 (3.2°) and brightness = 0.1 (0.027) stay below the gap. Geometric augmentations (flips, 90° rotations, small crops/zoom) are safe: a lesion has no "up" and its diagnosis does not depend on orientation. Strong colour augmentations are risky: colour (blue-white veil, red, dark brown) is a diagnostic feature, and jitter that shifts hue or brightness more than the real difference between benign and malignant can erase the signal or create misleading images. Only mild colour jitter (hue ≤ 0.02 , brightness ≤ 0.1) should be used. Augmentations must be approved by a clinical expert (a dermatologist), not only by the ML engineer.

A3.6 A lesion-level Dataset

```
In [26]: from torch.utils.data import DataLoader, Dataset

class LesionDataset(Dataset):
    """One item = one lesion: a dict with one image tensor per requested view, the integer label and the lesion_id."""

    VIEW_COLUMNS = {"derm": "derm_id", "clinical": "clinical_id"}

    def __init__(self, lesion_table, img_dir, label_col, transform, views=("derm", "clinical"), class_names=None):
        self.table = lesion_table.reset_index(drop=True)
        self.img_dir = Path(img_dir)
        self.label_col = label_col
```

```

        self.transform = transform
        self.views = views
        self.class_names = class_names or sorted(self.table[label_col].unique())
        self.class_to_idx = {name: i for i, name in enumerate(self.class_names)}
        self.paths = {p.stem: p for p in self.img_dir.rglob("*.jpg")}

    def __len__(self):
        return len(self.table)

    def __getitem__(self, i):
        row = self.table.iloc[i]
        item = {}
        for view in self.views:
            isic_id = row[self.VIEW_COLUMNS[view]]
            path = self.paths.get(isic_id)
            if path is None:
                raise FileNotFoundError(f"Missing image {isic_id} ({view} view of lesion {row['lesion_id']}) in {self.img_dir}")
            img = Image.open(path)
            img.draft("RGB", (256, 256)) # fast JPEG decoding at reduced size
            item[view] = self.transform(img.convert("RGB")) # transform applied to EACH view separately
        item["label"] = self.class_to_idx[row[self.label_col]]
        item["lesion_id"] = row["lesion_id"]
        return item

eval_tf = transforms.Compose([transforms.Resize((224, 224)), transforms.ToTensor()])

```

```

In [27]: # (a) one batch: shapes and lesion_ids
ds = LesionDataset(lesions, IMG_DIR, "diagnosis_1", eval_tf)
batch = next(iter(DataLoader(ds, batch_size=8, shuffle=False)))
for view in ds.views:
    print(view, tuple(batch[view].shape))
    assert tuple(batch[view].shape) == (8, 3, 224, 224)
assert list(batch["lesion_id"]) == lesions["lesion_id"].iloc[:8].tolist()
assert [ds.class_names[i] for i in batch["label"]] == lesions["diagnosis_1"].iloc[:8].tolist()
print("lesion_ids and labels match the table:", list(batch["lesion_id"][:3]), "...")

# clear error if an image is missing
broken = lesions.head(1).assign(derm_id="ISIC_DOES_NOT_EXIST")
try:
    LesionDataset(broken, IMG_DIR, "diagnosis_1", eval_tf)[0]

```

```
except FileNotFoundError as error:
    print("Missing file raises:", error)
```

```
derm (8, 3, 224, 224)
```

```
clinical (8, 3, 224, 224)
```

```
lesion_ids and labels match the table: ['IL_0000652', 'IL_0003176', 'IL_0004688'] ...
```

```
Missing file raises: Missing image ISIC_DOES_NOT_EXIST (derm view of lesion IL_0000652) in ..\data\milk10k\images
```

```
In [28]: # (b) one epoch over 800 lesions: class counts seen == class counts of the subset
subset = lesions.sample(min(800, len(lesions)), random_state=SEED)
ds_sub = LesionDataset(subset, IMG_DIR, "diagnosis_1", eval_tf, class_names=ds.class_names)

start = time.time()
seen = []
for b in DataLoader(ds_sub, batch_size=8, shuffle=True):
    seen += b["label"].tolist()
seen_counts = pd.Series([ds.class_names[i] for i in seen]).value_counts().sort_index()
expected_counts = subset["diagnosis_1"].value_counts().sort_index()
print(pd.DataFrame({"seen in epoch": seen_counts, "in subset": expected_counts}))
assert seen_counts.equals(expected_counts)
print(f"epoch over {len(subset)} lesions took {time.time() - start:.1f} s")
```

	seen in epoch	in subset
Benign	207	207
Indeterminate	22	22
Malignant	571	571

epoch over 800 lesions took 6.7 s

```
In [29]: # (c) image-level probabilities -> one prediction per lesion
def aggregate_predictions(image_probs, image_table, class_names):
    """Average the class probabilities of the images of each lesion and return (lesion_probs, lesion_prediction)."""
    probs = pd.DataFrame(image_probs, columns=class_names)
    probs["lesion_id"] = image_table["lesion_id"].values
    lesion_probs = probs.groupby("lesion_id").mean()
    return lesion_probs, lesion_probs.idxmax(axis=1)

# test with synthetic probabilities: lesion A = two images, lesion B = two images
fake_images = pd.DataFrame({"lesion_id": ["A", "A", "B", "B"]})
fake_probs = np.array([[0.9, 0.1, 0.0],
                       [0.3, 0.6, 0.1],
                       [0.2, 0.3, 0.5],
                       [0.3, 0.6, 0.1]], # A average = [0.6, 0.35, 0.05] -> class 0
```

```

[0.0, 0.1, 0.9]]) # B average = [0.1, 0.2, 0.7] -> class 2
lesion_probs, lesion_pred = aggregate_predictions(fake_probs, fake_images, LABELS)
display(lesion_probs)
assert np.allclose(lesion_probs.loc["A"], [0.6, 0.35, 0.05])
assert lesion_pred.to_dict() == {"A": "Benign", "B": "Malignant"}
print("aggregate_predictions test passed:", lesion_pred.to_dict())

```

	Benign	Indeterminate	Malignant
lesion_id			
A	0.6	0.35	0.05
B	0.1	0.20	0.70

aggregate_predictions test passed: {'A': 'Benign', 'B': 'Malignant'}

Answer A3.6(d). The correct unit of evaluation is the **lesion**: one lesion = one patient decision, so each lesion must be counted once, with its two views combined. Evaluating per image counts every lesion twice, which gives the two views of the same lesion double weight and makes confidence intervals look narrower than they are. If the split is also done per image, one view can be in train and the other in test, so the model is tested on lesions it has already seen and the score is inflated.